

17 - Prática Docker e Containers para Aplicações (III)

Thassiana C. A. Muller

Introdução

Containers Docker são uma forma simples e eficiente de empacotar, distribuir e executar aplicações com todas as suas dependências isoladas do sistema operacional. O Docker é uma das plataformas disponíveis para criar, gerenciar e executar esses containers, ele facilita o desenvolvimento, testes e implantação de aplicações e ferramentas em diferentes ambientes.

Imagens

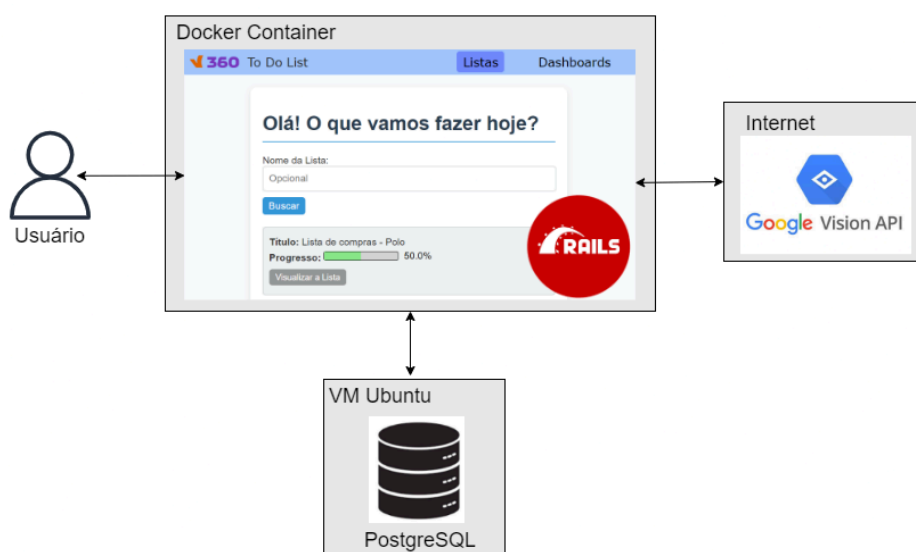
Contém código, bibliotecas, dependências, configurações, enfim, todo o necessário para executar uma aplicação. São feitas a partir de um arquivo chamado **Dockerfile** que contém as definições de como vai ser criado container. Podem ser armazenadas online no Docker Hub

Containers

São instâncias em execução de imagens, isolados do sistema host e de outros containers. Consomem muito menos recursos que máquinas virtuais já que uma máquina virtual precisa de todo o esqueleto de um SO para funcionar e o container apenas o que é necessário. Os containers têm seu próprio sistema de arquivos, rede e processos

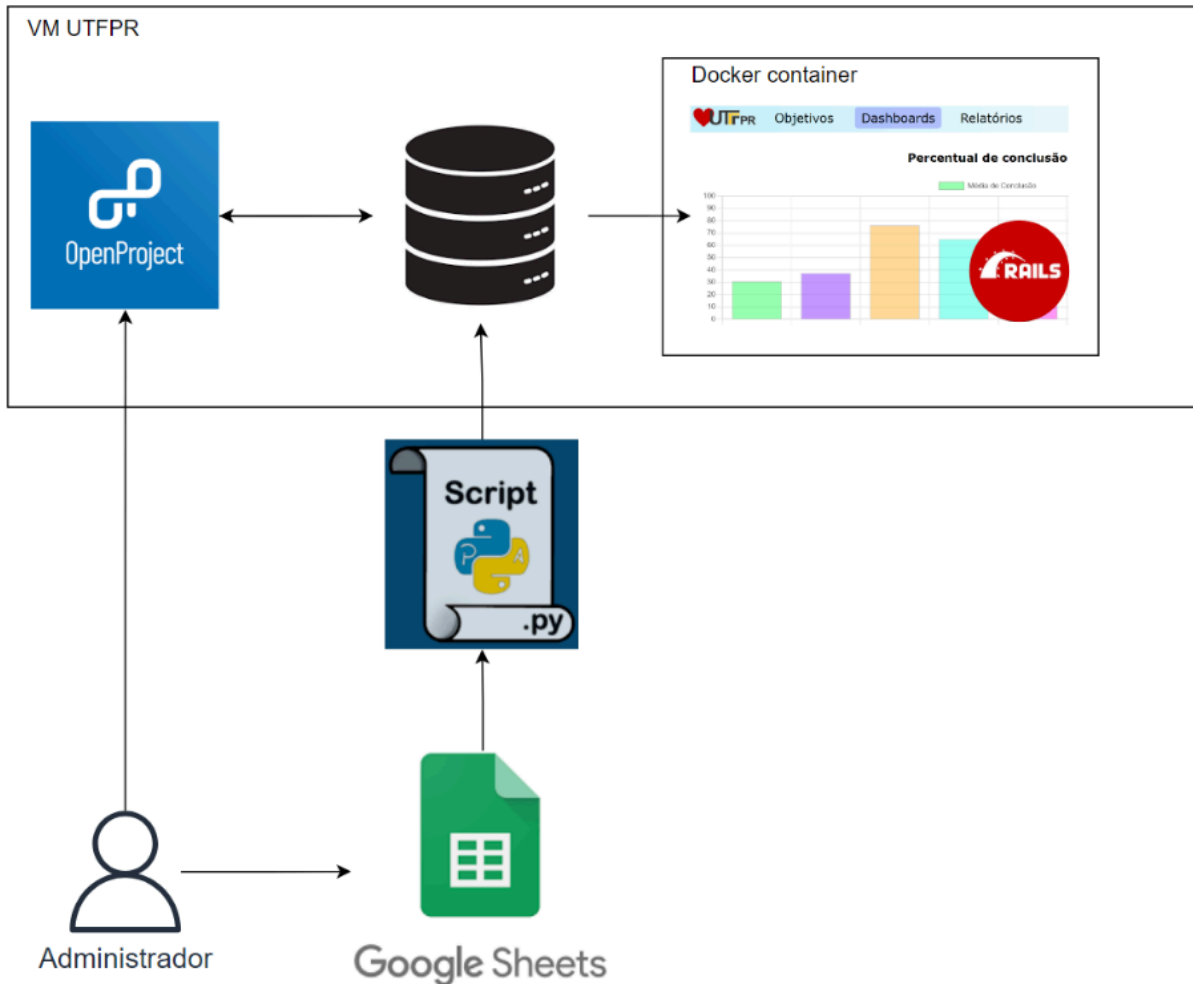
Docker - Exemplos práticos de aplicações

O container está hospedando uma aplicação completa, ele se comunica com APIs e banco de dados



<https://github.com/ThassiAmorim/ToDoList>

O container também pode ser parte de uma arquitetura distribuída de serviços. No exemplo abaixo, ele é responsável por hospedar o site do planejamento participativo da UTFPR-PB, facilitando a manutenção, visto que, caso ocorra alguma falha no container, é necessário apenas rebuildar a imagem corrigida, mantendo a integridade do resto da arquitetura.



Ciclo de vida de um container

Criação

```
thassi-ubuntu@Thassi:~$ docker container create --name teste_aula_thassi -it alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
f18232174bc9: Pull complete
Digest: sha256:a8560b36e8b8210634f77d9f7f9efd7ffa463e380b75e2e74aff4511df3ef88c
Status: Downloaded newer image for alpine:latest
36d4ac1e90d2af490e7067db45a5c63e47c0732894b6550a4dd2fdadebdbdd2b
```

- create = cria o container
 - --name = já atribui um nome
 - -it = modo interativo (mantem o terminal aberto e inicia um pseudo terminal interno do container)

- sh = inicia um shell quando o container iniciar

Visualização de todos os containers (inclusive os parados)

```
thassi-ubuntu@Thassi:~$ docker container ls -a
```

CONTAINER ID	IMAGE	PORTS	NAMES	COMMAND	CREATED	STAT
US						
36d4ac1e90d2	alpine		teste_aula_thassi	"sh"	About a minute ago	Crea
0f2161a8c1a9	apache/airflow:2.10.5		docker-airflow-airflow-worker-1	"/usr/bin/dumb-init ..."	6 weeks ago	Exit

- ls = lista os containers rodando
 - --a = mostra todos os containers

Start

```
thassi-ubuntu@Thassi:~$ docker container start teste_aula_thassi
teste_aula_thassi
thassi-ubuntu@Thassi:~$ docker container ls
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
36d4ac1e90d2	alpine	"sh"	3 minutes ago	Up 24 seconds		teste_aula_thassi

- start = roda/inicia o container

Acessando o sistema interno do container

```
thassi-ubuntu@Thassi:~$ docker container attach teste_aula_thassi
/ # ls
bin      etc      lib      mnt      proc     run      srv      tmp      var
dev      home    media    opt      root     sbin     sys      usr
```

- attach = se "conecta" ao container

Acessando com exec

```
thassi-ubuntu@Thassi:~$ docker container start teste_volume_alpine
teste_volume_alpine
thassi-ubuntu@Thassi:~$ docker container exec -it teste_volume_alpine sh
/ # ls
bin      dev      home    media    opt      root     sbin     sys      usr
data     etc      lib      mnt      proc     run      srv      tmp      var
```

- exec = acessa containers já startados

Saindo do container

```
thassi-ubuntu@Thassi:~$ docker container attach teste_aula_thassi
/ # ls
bin      etc      lib      mnt      proc     run      srv      tmp      var
dev      home    media    opt      root     sbin     sys      usr
/ # exit
thassi-ubuntu@Thassi:~$ docker container ls
thassi-ubuntu@Thassi:~$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
36d4ac1e90d2	alpine	"sh"	5 minutes ago	Exited (0) 6 seconds ago		teste_aula_thassi

Excluindo o container

```
thassi-ubuntu@Thassi:~$ docker container rm teste_aula_thassi
teste_aula_thassi
```

- rm = exclui o container

Pode se fazer todo o processo de criação, start e attach com um comando

```
thassi-ubuntu@Thassi:~$ docker container run -it --name teste_aula_thassi2 alpine sh
/ # ls
bin      etc      lib      mnt      proc     run      srv      tmp      var
dev      home    media    opt      root     sbin     sys      usr
/ #
```

- run = cria e inicia o container
 - -it = modo interativo (mantem o terminal aberto e inicia um pseudo terminal interno do container)
 - sh = inicia um shell quando o container iniciar

Manipulando arquivos

Copiando arquivos do meu sistema para o container

```
thassi-ubuntu@Thassi:~$ docker container cp Allumy teste_aula_thassi2:/
Successfully copied 2.05kB to teste_aula_thassi2:/
thassi-ubuntu@Thassi:~$ docker exec teste_aula_thassi2 ls
Allumy
bin
```

- cp = copia no sentido <origem> <destino>

Copiando arquivos do container para o sistema

```
thassi-ubuntu@Thassi:~$ docker container cp teste_aula_thassi2:/Allumy/thassi.txt ~/teste_copia_docker
Successfully copied 2.05kB to /home/thassi-ubuntu/teste_copia_docker
thassi-ubuntu@Thassi:~$ cat teste_copia_docker/thassi.txt
Amorim
```

Volumes

Usados para mapear diretórios do host diretamente ao container e salvar estados dos containers

Se eu criar um volume mapeado para um diretório do meu SO, o que eu modificar no diretório do container é refletido no diretório do SO e vice-versa

```
thassi-ubuntu@Thassi:~$ docker container run -v ~/bkp:/bkp/ --name teste_volume -it alpine sh
/ # cd bkp/
/bkp # ls
thassi.txt
/bkp # touch teste_volume.txt
```

- run = inicia o container

- -v = mapeia no sentido <host>:<container> uma pasta compartilhada entre container e host

```
thassi-ubuntu@Thassi:~/bkp$ ls
teste_volume.txt  thassi.txt
```

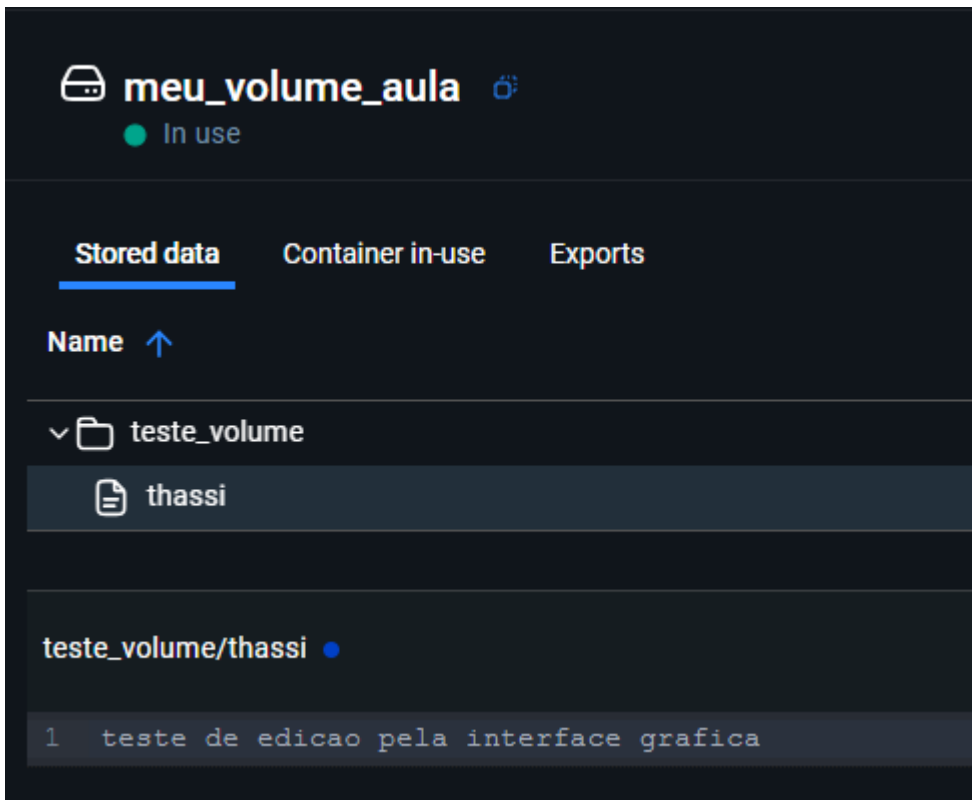
Isso é útil para salvar modificações feitas em um container pois, assim, mesmo que seja removido o container os arquivos alterados estão salvos em um volume no hospedeiro
Cuidar para modificar arquivos sensíveis no container pois isso irá refletir na máquina hospedeira

Na integração do wsl com o docker desktop, é possível visualizar os volumes rapidamente pela interface gráfica

cria o container vinculado ao volume “meu_volume_aula”

```
thassi-ubuntu@Thassi:~$ docker run -it --name teste_volume_alpine -v meu_volume_aula:/data alpine sh
/ # cd data
/data # mkdir teste_volume
/data # cd teste_volume/
/data/teste_volume # touch thassi
```

visualiza e altera o arquivo pela interface docker desktop



Interface do docker desktop na opção de visualização do volume “meu_volume_aula”

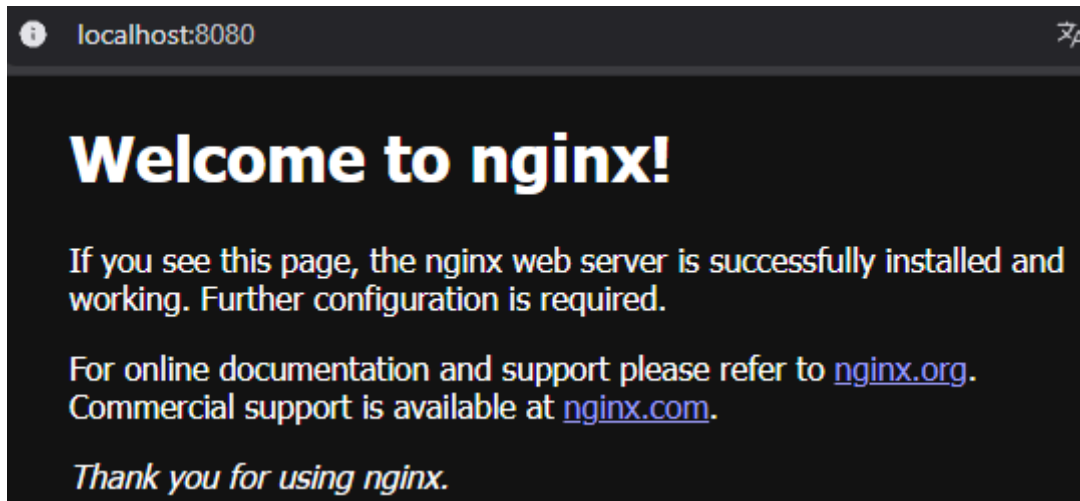
```
/data/teste_volume # cat thassi
teste de edicao pela interface grafica
```

Portas

Pode-se mapear, por exemplo, um container do nginx na porta 80 do container para a porta 8080 do hospedeiro

```
thassi-ubuntu@Thassi:~$ docker container run -p 8080:80 -d --name nginx nginx
205787fc9f18265a3dca70820eb14e73a5a54d0fa5fdae78e5a3d28c4b8a29d1
```

- run
 - -p = mapeia uma porta no sentido <host>:<container>



print do browser na porta 8080 após o start do container

Criando imagens a partir de containers - commit

Gosto da analogia do professor do curso em dizer que imagem está para classe e container está para o objeto instanciado. Dessa forma, imagem é o modelo que será usado para criar um container, ela contém as configurações e instruções para a criação do container

Para criar uma imagem a partir de um container usa-se commit

```
thassi-ubuntu@Thassi:~$ docker container run --name nginx-allumy-thassi -it alpine sh
/ # ls
bin      etc      lib      mnt      proc     run      srv      tmp      var
dev      home    media    opt      root     sbin     sys      usr
/ # mkdir allumy
/ # cd allumy/
/allumy # touch docker
/allumy # ls
docker
/allumy # echo "teste-thassi container para imagem" > docker
/allumy # cat docker
teste-thassi container para imagem
/allumy # exit

thassi-ubuntu@Thassi:~$ docker container commit nginx-allumy-thassi nginx-allumy-thassi-img
sha256:d543d52cde9f4e4fd19416def269846fa5030a97752d987bb61e193d3ed9cc89
```

- commit <container><nome_imagem> = cria uma nova imagem a partir do container

Salvando imagens .tar

```
thassi-ubuntu@Thassi:~/imagens_docker_lamia$ docker image save -o nginx-allumy-thassi.tar nginx-allumy-thassi-img
thassi-ubuntu@Thassi:~/imagens_docker_lamia$ ls
nginx-allumy-thassi.tar
thassi-ubuntu@Thassi:~/imagens_docker_lamia$ docker image rm nginx-allumy-thassi-img
Untagged: nginx-allumy-thassi-img:latest
Deleted: sha256:d543d52cde9f4e4fd19416def269846fa5030a97752d987bb61e193d3ed9cc89
Deleted: sha256:f39b131ad84ae1cad0eb4bc716ee1c11753b661d801b1c2b74975262c22676d8
thassi-ubuntu@Thassi:~/imagens_docker_lamia$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	a830707172e8	4 weeks ago	192MB
debian	bookworm-slim	8077017cbbd1	2 months ago	74.8MB
postgres	13	ebcc4a15b2eb	2 months ago	423MB
alpine	latest	aded1e1a5b37	3 months ago	7.83MB
apache/airflow	2.10.5	b6e72f8ddc68	3 months ago	1.56GB

- image save -o <saida><imagem>= salva a imagem

```
thassi-ubuntu@Thassi:~/imagens_docker_lamia$ docker image load -i nginx-allumy-thassi.tar
15d448fc01ba: Loading layer 4.096kB/4.096kB
Loaded image: nginx-allumy-thassi-img:latest
thassi-ubuntu@Thassi:~/imagens_docker_lamia$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx-allumy-thassi-img	latest	d543d52cde9f	9 minutes ago	7.83MB
nginx	latest	a830707172e8	4 weeks ago	192MB
debian	bookworm-slim	8077017cbbd1	2 months ago	74.8MB
postgres	13	ebcc4a15b2eb	2 months ago	423MB
alpine	latest	aded1e1a5b37	3 months ago	7.83MB
apache/airflow	2.10.5	b6e72f8ddc68	3 months ago	1.56GB

- image load = carrega uma imagem
 - -i = para arquivos tar

Testando a imagem carregada do tar

```
thassi-ubuntu@Thassi:~/imagens_docker_lamia$ docker container run -it --rm nginx-allumy-thassi-img:latest
/ # cd allumy/
/allumy # ls
docker
/allumy # cat docker
teste-thassi container para imagem
/allumy #
```

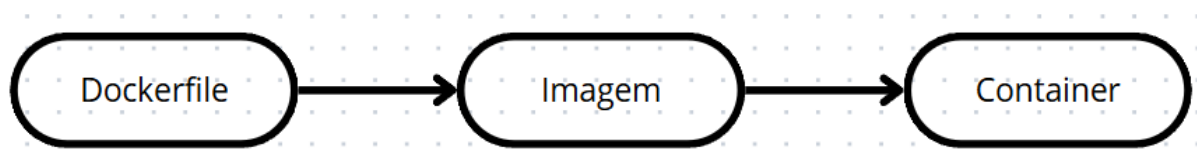


Fonte: autoria própria

Comandos úteis - imagens

- image inspect = traz informações de uma imagem
- image prune = remove imagens que não estão associadas a nenhum container
- image rm = remove imagem
- image pull = pega a imagem do docker hub
- image push = envia imagem pro docker hub (se estiver devidamente autenticado)

Dockerfile



Fonte: autoria própria

Um dockerfile é um arquivo para criação de imagens

```
thassi-ubuntu@Thassi:~/docker-lamia$ sudo nano Dockerfile
thassi-ubuntu@Thassi:~/docker-lamia$ cat Dockerfile
FROM ubuntu
RUN apt-get update && apt-get -y install vim
thassi-ubuntu@Thassi:~/docker-lamia$ docker image build -t meu_ubuntu .
[+] Building 34.1s (7/7) FINISHED
```

- image build = cria a imagem
 - -t = define um nome

```
thassi-ubuntu@Thassi:~/docker-lamia$ docker run -it --rm meu_ubuntu
root@89538cecf66a:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@89538cecf66a:/# vim teste
root@89538cecf66a:/#
```

Comandos úteis - System

```
thassi-ubuntu@Thassi:~$ docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	17	7	9.114GB	7.392GB (81%)
Containers	13	0	797.8MB	797.8MB (100%)
Local Volumes	5	1	71.21MB	71.21MB (99%)
Build Cache	415	0	10.89GB	10.89GB

- system df = lista o espaço ocupado por imagens, containers, volumes e cache
- **docker system prune** remove
 - containers parados
 - volumes não utilizados
 - redes não utilizadas
 - imagens não utilizadas
- pode especificar tipo: docker image prune

Exercicio

- a) Rodar em modo iterativo um container a partir da imagem do ubuntu, usando apt-get instalar o NGINX, inicializar o servidor NGINX

```
thassi-ubuntu@Thassi:~$ docker run -it --name exercicio_ubuntu ubuntu sh
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
0622fac788ed: Already exists
Digest: sha256:6015f66923d7afbc53558d7ccffd325d43b4e249f41a6e93eef074c9505d2233
Status: Downloaded newer image for ubuntu:latest
# apt-get update
```

```
# apt-get install nginx
```

```
# service nginx start
* Starting nginx nginx
# service nginx status
* nginx is running
```

- b) Com o comando commit, gerar uma imagem chamada ubuntu-allumy (coloquei ubuntu_nginx_img)

```
# thassi-ubuntu@Thassi:~$ docker container commit exercicio_ubuntu ubuntu_nginx_img
sha256:3981a63aefc9c0cfe3e1048152cdaac7c1b2d897e0467852b41e9bba708c8165
```

- c) Exportar a imagem para o arquivo ubuntu-allumy.tar através do comando save

```
thassi-ubuntu@Thassi:~$ docker image save -o ubuntu-allumy.tar ubuntu_nginx_img
```

- d) remover a imagem ubuntu-allumy (no caso ubuntu_nginx_img)

```
thassi-ubuntu@Thassi:~$ docker container rm -f exercicio_ubuntu
exercicio_ubuntu
thassi-ubuntu@Thassi:~$ docker image rm ubuntu_nginx_img
Untagged: ubuntu_nginx_img:latest
Deleted: sha256:3981a63aefc9c0cfe3e1048152cdaac7c1b2d897e0467852b41e9bba708c8165
Deleted: sha256:915c27a6864469158566dce86417f23fa45e886b02540272bcf81bf03741015a
```

- e) importar a imagem com o ubuntu-allumy.tar através do comando load

```
thassi-ubuntu@Thassi:~$ docker image load -i ubuntu-allumy.tar
e79d59e98c16: Loading layer [=====>] 57.44MB/57.44MB
Loaded image: ubuntu_nginx_img:latest
```

- f) rodar em um container

```
thassi-ubuntu@Thassi:~$ docker container run -it ubuntu_nginx_img
# service nginx status
* nginx is not running
# service nginx start
* Starting nginx nginx
# service nginx status
* nginx is running
#
```

Conclusão

Imagens e container são excelentes alternativas para rodar aplicações de maneira prática e isolada, são eficientes e leves no quesito recursos e podem ser facilmente compartilhados, além de garantirem a integridade e manutenção dos sistemas de softwares.